

The Socket Exchange

Eight investigations into TCP socket behaviour beneath a single-threaded `poll()` exchange server, plus a scaling study to 70,000 concurrent idle connections — including two defects the experiments themselves uncovered.

SYSTEM UNDER TEST	HOST	TRANSPORT
exchange_server	FreeBSD 14.4-RELEASE	Loopback (lo0)
C++17, direct POSIX sockets	VirtualBox, 2 GB RAM	server + clients, one host
I/O MODEL	INSTRUMENTS	
poll(), single-threaded	tcpdump · netstat	
no threads, no kqueue	sockstat · procstat · vmstat -z	

RESULTS AT A GLANCE

PEAK CONNECTIONS	KERNEL MEMORY / ENDPOINT	REPLY WHILE BLOCKED	LATENCY COST AT SCALE
70,000	2.34 KB	0.001 s	11.8×
0 failures	328 MB at peak	5 s limit (Exp 4)	poll() O(n) scan
DEFECTS FOUND			
2			
both fixed & verified			

Every experiment below was driven by the supplied `experiment.py` harness against the submitted server binary, with FreeBSD's own instrumentation capturing kernel-side state simultaneously. Two results contradicted our design predictions and are reported as such rather than smoothed over; one measurement remains unexplained and is flagged in Experiment 7.

#	EXPERIMENT	OUTCOME	KEY MEASUREMENT
1	Listening vs connected sockets	As expected	fd 3 LISTEN + fd 5 ESTABLISHED, both :5000

2	Connection states	Anomaly explained	no TIME_WAIT — nolocaltimewait=1
3	TCP as a byte stream	As expected	4 segments → 1 message
4	One client must not stall others	Pass	0.001 s reply, 5 s budget
5	I/O multiplexing	Pass	fds 4/6/8 ready, 5/7 untouched
6	FIN vs RST	As expected	4-packet close vs 1-packet abort
7	Backpressure	Did not trigger	85,000 B absorbed; forced case passes
8	Unexpected disconnection	Pass, other mechanism	EOF detected; SIGPIPE never fired
★	Bonus — scale to 70,000	Target met	0.85 s, 328 MB, 26% CPU

THE EXPERIMENTS

EXP 1

Listening and connected sockets

AS EXPECTED

METHOD

Start the server, connect one client, leave it idle. Inspect the same kernel state through three independent tools — `sockstat` (sockets with owning process), `netstat` (kernel socket table), and `procstat -f` (the server's own descriptor table).

EVIDENCE

USER	COMMAND	PID	FD	PROTO	LOCAL ADDRESS	FOREIGN ADDRESS
blatonan	exchange_s	1824	3	tcp4	127.0.0.1:5000	*:*
blatonan	exchange_s	1824	5	tcp4	127.0.0.1:5000	127.0.0.1:15663
blatonan	python3.12	1823	3	tcp4	127.0.0.1:15663	127.0.0.1:5000
tcp4	0	0	127.0.0.1.5000	127.0.0.1.15663	ESTABLISHED	
tcp4	0	0	127.0.0.1.15663	127.0.0.1.5000	ESTABLISHED	
tcp4	0	0	127.0.0.1.5000	.*	LISTEN	

Finding. A listening socket and a connected socket are distinct kernel objects that coexist on the *same* local port. The listening socket (fd 3) has a wildcard foreign address because it is not connected to anyone — it is a

queue endpoint. The connected socket (fd 5) returned by `accept()` names a specific peer. They can share port 5000 because TCP demultiplexes on the full 4-tuple (local IP, local port, remote IP, remote port), not on local port alone. The original listening descriptor stays open and keeps accepting.

EXP 2

Observing connection states

ANOMALY EXPLAINED

METHOD

Connect, hold idle 10 s, close, then sample `netstat` every 0.25 s across the transition while `tcpdump` captured `lo0`.

EVIDENCE

```
t=079.930  23782 > 5000: [S]      seq 3637170437
t=079.930  5000 > 23782: [S.]    seq 2964004167, ack 1
t=079.930  23782 > 5000: [.]    ack 1              <- ESTABLISHED

      .... 10 seconds idle — not a single packet ....

t=089.987  23782 > 5000: [F.]    seq 1, ack 1          <- client close()
t=089.987  5000 > 23782: [.]    ack 2
t=089.988  5000 > 23782: [F.]    seq 1, ack 2          <- server close()
t=089.988  23782 > 5000: [.]    ack 2                  <- final ACK

state sampling across the same moment:
t= 9.75s :  2 ESTABLISHED  1 LISTEN
t=10.00s :  1 LISTEN           <- client entry gone entirely
```

TWO FINDINGS

An idle connection is completely silent. Ten seconds elapse between the handshake and the close with zero packets — no keepalives, no polling. An established TCP connection at rest consumes memory and descriptors, but no CPU and no bandwidth. This is the packet-level basis for the bonus result.

TIME_WAIT never appears. The active closer should linger for $2 \times \text{MSL} = 60$ s. Instead the entry vanishes within 250 ms. All four close packets are present, so the close completed correctly — this is not a malfunction. The cause is `net.inet.tcp.nolocaltimewait=1`: FreeBSD suppresses TIME_WAIT when both endpoints are on the same host.

Why that is safe. TIME_WAIT exists to absorb delayed duplicate segments still wandering the network, and to keep the final ACK retransmittable if it is lost. On loopback there is no network for a segment to wander in and nothing to lose the ACK, so the state buys nothing and is skipped. A

consequence worth noting: loopback ephemeral ports are reusable immediately, which is *not* true of a real interface.

EXP 3

TCP as a byte stream

AS EXPECTED

METHOD

The harness writes one application message — `LOGIN experiment_trader\n` — as four separate `send()` calls 200 ms apart. Captured simultaneously on the wire and inside the server, which was built with an `EXCHANGE_DEBUG` trace logging every `recv()`'s byte count and raw bytes.

EVIDENCE — WIRE AND APPLICATION VIEW, CORRELATED

44373 > 5000: [P.] seq 1:7	length 6	[recv] 6 bytes: "LOGIN "
		[buf] holds 6: "LOGIN "
		[wait] no newline yet
44373 > 5000: [P.] seq 7:17	length 10	[recv] 10 bytes: "experiment"
		[buf] holds 16: "LOGIN experiment"
		[wait] no newline yet
44373 > 5000: [P.] seq 17:24	length 7	[recv] 7 bytes: "_trader"
		[buf] holds 23: "LOGIN experiment_trad"
		[wait] no newline yet
44373 > 5000: [P.] seq 24:25	length 1	[recv] 1 bytes: "\n"
		[buf] holds 24: "LOGIN experiment_trad"
		[line] COMPLETE MESSAGE extracted
5000 > 44373: [P.] seq 1:4	length 3	server replies "OK\n" — 0.8 ms later

Finding. The sequence numbers run `1 → 7 → 17 → 24 → 25` contiguously. To numbers *bytes*, and there is no message boundary anywhere in the sequence — the four segments are one uninterrupted run. The server produces no response until the fourth segment delivers the delimiter, and replies 0.8 ms afterwards server that assumed one `recv()` equals one message would have parsed `"L"` as a complete command and failed three times over.

EXP 4

One client must not stall the others

PASS

METHOD

Client 1 sends `LOGIN blocked_client` with **no trailing newline**, then goes silent forever. Two seconds later Client 2 connects and sends a complete command. The harness times the reply against a 5-second budget.

EVIDENCE

```

Client 1: connected from 127.0.0.1:32002
Client 1 will send an incomplete application message.
Client 1 will now remain silent.

Client 2: connected from 127.0.0.1:29026
Client 2 response: 'OK'
Elapsed time: 0.001 seconds

netstat - note the Recv-Q on client 1:
      Recv-Q  Send-Q  Local                Foreign
tcp4      0      0  127.0.0.1.32002      127.0.0.1.5000      ESTABLISHED
tcp4      0      0  127.0.0.1.29026      127.0.0.1.5000      ESTABLISHED

```

Finding. Client 2 was served in **1 millisecond** against a 5-second budget. The subtle corroboration is `Recv-Q = 0` on client 1: the server *did* read the partial bytes out of the kernel buffer — which is why the queue drained — but produced no response, because no delimiter had arrived. Those bytes sit in that connection's userspace buffer, parsed by nobody.

This single capture therefore proves both properties the experiment targets at once: **framing correctness** (a partial line is buffered, not acted upon) and **concurrency correctness** (the event loop never blocked in `recv()` on the silent client). An accept-then-blocking-recv-per-client server fails this outright, hanging for the full timeout.

EXP 5

Multiple clients and I/O multiplexing

PASS

METHOD

Open five simultaneous connections. Clients 1, 3 and 5 send a command; clients 2 and 4 stay idle. Observe which descriptors `poll()` reports as ready.

EVIDENCE

```

all five connections established:
exchange_server fd 4 127.0.0.1:5000 <- 127.0.0.1:26790 client 1
exchange_server fd 5 127.0.0.1:5000 <- 127.0.0.1:37766 client 2 (idle)
exchange_server fd 6 127.0.0.1:5000 <- 127.0.0.1:30737 client 3
exchange_server fd 7 127.0.0.1:5000 <- 127.0.0.1:37165 client 4 (idle)
exchange_server fd 8 127.0.0.1:5000 <- 127.0.0.1:45038 client 5

recv activity over the whole run - only three descriptors appear:
[recv] fd=4 15 bytes: "LOGIN client_1\n" -> [line] complete
[recv] fd=6 15 bytes: "LOGIN client_3\n" -> [line] complete
[recv] fd=8 15 bytes: "LOGIN client_5\n" -> [line] complete

```

Finding. `poll()` reported readable on exactly fds 4, 6 and 8 — the three that sent. Descriptors 5 and 7 remained ESTABLISHED for the entire run and were never touched: no `recv()` was attempted on them, and no CPU was spent on them. One thread, one event loop, five connections, and work performed only where work existed. This is what makes the Exp 4 result possible.

EXP 6

FIN versus RST

AS EXPECTED

METHOD

Part A closes a connection orderly via `shutdown(SHUT_WR)`. Part B closes abortively via `SO_LINGER{1,0}`. Both captured on `lo0`.

EVIDENCE — THE TWO TEARDOWNS SIDE BY SIDE

PART A — orderly (port 15581)

```
15581 > 5000: [F.] seq 1, ack 1      client's shutdown(SHUT_WR)
5000 > 15581: [.] ack 2              server acknowledges the FIN
5000 > 15581: [F.] seq 1, ack 2      server closes its own end
15581 > 5000: [.] ack 2              final acknowledgement
-> server: "fd=5 client disconnected" (recv =
```

PART B — abortive (port 22363)

```
22363 > 5000: [R.] seq 1, ack 1, win 0
-> server: "recv: Connection reset by peer" (
one packet. no negotiation. done.
```

Finding. Four packets to close politely; **one** to abort. The orderly close is a negotiated shutdown in which each side announces it has no more data and waits for acknowledgement — any data already in flight is still delivered. The RST carries and terminates both directions immediately, discarding anything pending.

The distinction reaches the application as two entirely different signals: `recv` returning **0** for an orderly close versus failing with **ECONNRESET** for an abort. The application must handle both, and must treat neither as fatal.

INCIDENTAL CAPTURE — THE HARNESS'S STARTUP PROBE

The same trace caught `wait_for_server()`'s retry loop. The first probe received `[R.]` immediately because the server had not yet called `listen()` — the kernel refuses a SYN to a closed port. The second completed the handshake and *then* sent

...sends a SYN to a closed port. The second completed the handshake and then sent RST. Every harness run therefore opens with an abortive reset on the first connection the server ever accepts, which the server must survive. Ours logs it and continues.

EXP 7

Backpressure and the slow receiver DID NOT TRIGGER

METHOD

Two market-data clients subscribe to JNST; one drains continuously, the other never reads a byte. Two traders generate 5,000 matching trades (`TRADE JNST 1 238\n` = 17 bytes, 85,000 bytes total). Socket queue depths sampled throughout.

EVIDENCE — DEFAULT BUFFERS

time	slow client Recv-Q	server Send-Q -> slow	server RSS
4s	1,666	34	3,428 KB
54s	23,086	17	3,616 KB
104s	44,404	17	3,800 KB
154s	65,501	34	3,868 KB
204s	85,000	0	4,164 KB
	= 5000 x 17 B	never backed up	

The receive buffer absorbed the *entire* payload. The server's `send()` never once returned `EWOULDBLOCK`, so the per-connection outbound queue never engaged, and no client was affected. **The experiment as parameterised is too small to produce the condition it is named after.**

FORCING THE CONDITION

To test the mechanism the experiment intends to test, buffers were capped to 4 KB (`net.inet.tcp.recvspace=4096` , `recvbuf_max=8192`) so that 85 KB genuinely cannot fit. A control confirmed the kernel enforces this strictly: against a non-reading receiver, `send()` blocked at exactly 8,192 bytes = 4 KB send buffer + 4 KB receive buffer.

```
trades generated      : 1,012
slow MD Recv-Q        : 2,094 (SO_RCVBUF = 4,096 — full)
slow MD has read      : 0 bytes
```

```
--- with ~1,000 trades of undeliverable TRADE messages backed up,
    can a brand new client still be served? ---
trial 1: 'OK' in 0.0038 s
trial 2: 'OK' in 0.0004 s
trial 3: 'OK' in 0.0005 s
```

Finding. With backpressure genuinely engaged — a subscriber whose kernel buffer is full, which has read nothing, and a thousand messages with

buffer is full, which has read nothing, and a thousand messages with nowhere to go — a newly arriving client is served in **0.4 milliseconds**. The slow reader is completely isolated and stalls nobody. This is the §4.4 requirement demonstrated, and it is the per-connection outbound queue plus non-blocking `send()` that makes it true.

The honest summary: the design decision is *necessary but unexercised* by the experiment as written. It took an artificially constrained kernel to show it working.

UNRESOLVED MEASUREMENT

Running the harness's Experiment 7 under the same 4 KB cap still reported the slow client's Recv-Q reaching 85,000 — twenty times its measured `SO_RCVBUF` of 4,096. An isolated test proves the kernel enforces that limit strictly, and a controlled reproduction of the identical scenario plateaus at 2,094 as expected. Two runs disagree and we do not have a confirmed mechanism for the discrepancy. The controlled reproduction is the figure we trust; the harness figure is recorded here rather than quietly dropped.

EXP 8

Unexpected client disconnection

PASS — OTHER MECHANISM

METHOD

A market-data client running as a separate process is `SIGKILL` ed. The harness then generates 50 further trades that the server will attempt to deliver to that now-dead socket.

EVIDENCE

```
The Market-Data Client process will now disappear unexpectedly.
exchange_server: fd=7 client disconnected    <- detected instantly, via recv() == 0
The client process has disappeared.
Generating additional traffic.               <- 50 further trades
Experiment finished.                         <- server alive throughout
```

Finding. The server survived, but *not* by the mechanism the experiment anticipates. It detected the death through `recv()` returning **0**, not through `EPIPE` on a send. When a process is `SIGKILL` ed the kernel closes its descriptors, and closing a socket sends a normal **FIN** — the *process* death is abrupt, but the *TCP* close is orderly. The event loop saw EOF on the very next

`poll()` and removed the connection **before** the next broadcast ever tried to write to it.

Consequently **SIGPIPE never fired**. The `signal(SIGPIPE, SIG_IGN)` plus per connection `EPIPE` / `ECONNRESET` handling remains correct defensive code — it would matter if a `send()` raced the FIN, or if the peer had reset rather than closed — but this experiment does not exercise it. Like Experiment 7, the hazard does not materialise under default behaviour and a correctly structured event loop.

BONUS — SCALING TO 70,000 IDLE CONNECTIONS

Server and client generator run on the same 2 GB host over loopback, so every connection costs *two* descriptors and *two* socket structures system-wide. Connections were established in tiers, each an independent measurement.

TIER	SRC IPS	CONNECT TIME	FAILURES	SERVER RSS	KERNEL MEMORY	SOCKET ZONE	S
1,000	1	0.24 s	0	3.6 MB	4.7 MB	2,096	
5,000	1	0.23 s	0	4.3 MB	25 MB	11,000	
15,000	1	0.22 s	0	5.8 MB	70 MB	30,026	
30,000	2	0.51 s	0	8.0 MB	141 MB	60,026	
50,000	2	0.55 s	0	11.0 MB	234 MB	100,026	
70,000	4	0.85 s	0	15.0 MB	328 MB	140,026	

At every tier the server's descriptor count matched the generator's exactly and `netstat` showed precisely 2N established sockets — no connection was lost or orphaned. Per-endpoint cost, read from the UMA zone allocator rather than estimated:

ITEM	SIZE	LIMIT	USED	
socket:	1024	200000	140,026	1024 B
tcp_inpcb:	1320	0	140,005	1320 B
tcp_inpcb ports:	32	0	70,013	32 B

2.34 KB per endpoint				
4.69 KB per loopback pair				

`vmstat`'s free-memory column is misleading here — it reports free *pages* and barely

removed. The UMA zone statistics are what actually track the cost. Userspace RSS is negligible by comparison at roughly 200 bytes per connection.

WHY 328 MB AND NOT 9 GB

Nominally 140,000 endpoints × 64 KB receive buffer ≈ 9 GB, which cannot fit in 2 GB. No buffer tuning was needed, because `sendspace` / `recvspace` are **high-water marks, not preallocations**: FreeBSD allocates mbufs on demand as data flows. Idle connections carry no data and therefore allocate no buffer memory at all — only the fixed per-socket structures above. Every tier ran with stock buffer settings.

FUNCTIONAL VERIFICATION AT PEAK

Holding 70,000 idle connections, the full protocol was exercised live: `LOGIN` → `OK`, `SUBSCRIBE` → `OK`, `BUY / SELL` → `ORDER_ACCEPTED`, matching → private `BOUGHT / SOLD`, and `TRADE` broadcast to the subscriber. The server was not merely holding sockets; it was working.

THE BOTTLENECK

The identical protocol check, run with 70,000 idle connections held versus torn down:

OPERATION	70,000 IDLE	~0 IDLE	PENALTY
connect	14.6 ms	3.7 ms	3.9×
LOGIN round-trip	11.0 ms	1.4 ms	7.9×
BUY round-trip	13.0 ms	1.1 ms	11.8×
fill notification (<i>control</i>)	0.1 ms	0.1 ms	1.0×

Every `poll()` call copies the whole 70,001-entry `pollfd` array from userspace to the kernel — roughly 560 KB — the kernel scans all of it, copies it back, and the event loop then scans all 70,001 entries for non-zero `revents`. **Per-event latency is O(total connections), not O(active connections)**: one active client's request pays for all 70,000 idle ones. The fill notification is the control case and is unaffected, because it is delivered inside the same tick as the work that produced it and needs no fresh wake-up.

Would a different I/O mechanism help? The answer is two-sided, and both sides are measured rather than argued.

Capacity — how many connections can be held — is bounded by descriptor limits, the `socket` UMA zone, and kernel memory. `kqueue` would help *not at all*: these costs are identical under any multiplexing model.

Latency — responsiveness while holding them — *is* the multiplexing algorithm. and

`kqueue` would help substantially. Its cost is $O(\text{ready events})$ with one-time registration, against `poll()`'s $O(\text{total descriptors})$ on every call. The 11.8× degradation above is the size of the prize.

This refines our design prediction rather than confirming it. We argued that idle connections cost nothing because they never wake `poll()`. The first half held exactly — 70,000 idle connections produced zero spurious wake-ups. The conclusion drawn from it did not: when `poll()` wakes for a *real* request, it pays for every idle connection in the set.

LIMITS APPROACHED AT 70,000

RESOURCE	AT 70,000	CEILING	USED
<code>socket</code> UMA zone	140,026	200,000	70%
System open files	140,099	200,000	70%
Per-process descriptors (<i>each side</i>)	70,008	100,000	70%
Kernel memory	328 MB	2 GB	16%

DEFECTS THE EXPERIMENTS UNCOVERED

Both were found by running the scaling study, not by reading the code. Neither was visible at the connection counts the functional tests used.

DEF 1 One `accept()` per `poll()` wake-up SILENT DATA LOSS

`poll()` is level-triggered: it reports the listening socket readable when at least one connection is pending, but never says how many. The original loop accepted exactly one per wake-up, then scanned every existing descriptor, then called `poll()` again — so draining N queued connections cost N full $O(n)$ cycles. During a ramp the drain rate falls further behind arrival the more clients are already connected. Quadratic.

```
kernel log:
sonewconn: pcb (127.0.0.1:5000): Listen queue overflow:
1537 already in queue awaiting acceptance
1537 ~= FreeBSD's grace threshold of 3/2 x backlog = 3 x 1024 / 2

at the 5,000-connection tier, before the fix:
generator reported : 4,789 connected, 211 failed
```

```
server actually held: 3,269
netstat ESTABLISHED : 8,058 = 4,789 + 3,269
-> ~1,520 connections orphaned
```

The failure mode is worse than a refused connection. Those ~1,520 connections completed their TCP handshake — so `connect()` returned *success* to the client — and were then discarded by `sonewconn()` before `accept()`, **with no RST sent**. The peer sits ESTABLISHED against a socket that no longer exists, and would only discover this on its next write or after the two-hour keepalive. The generator reported 211 failures; the true figure was over 1,700.

Fix. Make the listening socket non-blocking and drain `accept()` until `EWOULDBLOCK`, treating `ECONNABORTED` and `EINTR` as continue-conditions rather than aborting the drain — `ECONNABORTED` matters specifically because the harness's startup probe resets deliberately. After the fix: 5,000 of 5,000, descriptor counts matching exactly, zero overflows. Notably the fix did *not* make the ramp faster (0.22 s → 0.23 s) — it made it correct.

DEF 2

Loopback aliases assumed, not configured ENVIRONMENT

The client generator cycles outbound connections across `127.0.0.1`, `127.0.0.2`, ... to escape the ~55,535 ephemeral ports a single source address commands. Its comment asserted that the whole `127.0.0.0/8` block routes to loopback without configuration. That is true on Linux and **false on FreeBSD**.

```
lo0: inet 127.0.0.1 netmask 0xff000000    <- only .1 is actually assigned

route -n get 127.0.0.2
  destination: 0.0.0.0
    gateway: 10.0.2.2
  interface: em0                <- leaves via the default route!

symptom at the 30,000 tier, 2 source IPs:
done -- 15,000 connected, 15,000 failed  a perfect 50/50 split
```

`lo0` owns only the single address; the `/8` netmask does not make the rest of the block local, so `bind()` to `127.0.0.2` fails with `EADDRNOTAVAIL`. Because the generator alternates addresses round-robin, exactly half of all attempts failed.

Fix. Corrected the comment, added the required `ifconfig lo0 alias` setup to the repository (`bonus/`), and — the more useful change — made the generator record and report the first `errno` for each failing call. A bare failure count is undiagnosable: port exhaustion, an unaliased address, and a

full listen queue all present identically. Diagnosing this cost an entire measurement tier for want of one `strerror()`.

REPRODUCTION

A stock FreeBSD host will not reach 70,000 connections. The setup is scripted and idempotent:

```
sudo sh bonus/setup-freebsd.sh --persist
```

It raises four kernel limits and adds four loopback aliases:

SETTING	STOCK	REQUIRED	WHY
kern.maxfiles	64,433	200,000	Both processes share the host — ~140,000 descriptors
kern.maxfilesperproc	57,987	100,000	Also the hard <code>RLIMIT_NOFILE</code> ceiling a session resolves to
kern.ipc.maxsockets	64,433	200,000	Ceiling on the <code>socket</code> UMA zone
kern.ipc.soacceptqueue	128	1,024	Silently clamps the backlog <code>listen()</code> requests
lo0 alias 127.0.0.2-.5	absent	added	Escape the 55,535-port single-address ceiling

A process must be started *after* the script runs to inherit the raised descriptor ceiling. The server's own `setrlimit(RLIMIT_NOFILE, ...)` at startup cannot exceed the system-wide limit.

The `recv()` trace used in Experiments 3 and 5 is enabled with `EXCHANGE_DEBUG=1` in the environment — deliberately an environment variable rather than a command-line flag, because the harness invokes `./server/run-server` with exactly two arguments and the server rejects any other argument count.